

Agent Orchestration

Model Selection Strategy

Day 1 - Session 1, Topic 2

What We'll Cover

1. Models, agents, and selection tradeoffs
2. Task evidence, model tier, and effort
3. Completed-task economics
4. Demo: Model Selection Matrix
5. Demo: Evidence-Based Escalation

Model, Agent, and Orchestrator

Term	Role in agentic development
Model	Produces reasoning and responses from the context it receives.
Agent	Combines a model with a goal, instructions, tools, and an action loop.
Orchestrator	Assigns work, selects resources, coordinates dependencies, and checks outcomes.

The model is one component of an agent, not the entire agentic system.

Why Select Models?

Models and configurations make different tradeoffs:

- Capability for ambiguity, breadth, and difficult reasoning
- Speed and responsiveness
- Usage cost and resource consumption
- Performance with the required context and tools

Select for reliable task completion, not a permanent “best model” ranking.

Route from Evidence

Capability, speed, and price are constraints, not labels

Start with the Task Card

Score the work before opening the model selector.

1. Complexity and repository breadth
2. Uncertainty and missing context
3. Blast radius and reversibility
4. Verification cost and available checks
5. Latency sensitivity and completion value

Model Selection Is Not Risk Control

A stronger model may reduce some execution errors.

It does not replace scoped permissions, security gates, or human approval.

High-blast-radius work needs stronger controls regardless of model choice.

Capability answers "Can it reason about this?" Governance answers "May it act?"

Governance or Capability?

A strong model proposes a high-blast-radius change. Which factor is governance?

- A) Context capacity
- B) Approval authority
- C) Debugging skill
- D) Response speed

Chat Check: Type your answer (A, B, C, or D) in the chat window now.

Governance or Capability: The Answer

B - Approval authority

Governance determines whether the model may act. Capability, context, and speed do not grant permission for a high-risk change.

Evidence-Based Model Routing (SaaS)

SaaS: Software as a Service

REAL-WORLD SCENARIO

SaaS (Vercel - AI Gateway):

- Teams can route by measured uptime and latency, set provider order, and define model fallbacks.
- Model metadata exposes pricing and reasoning controls, while scoped budgets cap spend before each request.

Evidence-Based Model Routing (Financial Services)

SKIP

REAL-WORLD SCENARIO

Financial services (Block - goose):

- Block's goose supports multiple model providers so teams can switch according to capability and budget needs.
- Local execution can write and test code while permissions and security controls govern tool access.

Current Claude Model Baseline

Model	Typical starting point	Relative speed
Haiku 4.5	Routine, high-volume, cost-sensitive work	Fastest
Sonnet 5	Everyday coding with speed and capability	Fast
Opus 5	Complex agentic coding and enterprise work	Moderate
Fable 5.1	Demanding, long-horizon agentic work	Slower

Verified 2026-09-13. Availability and product details can change.

Effort Is a Separate Lever

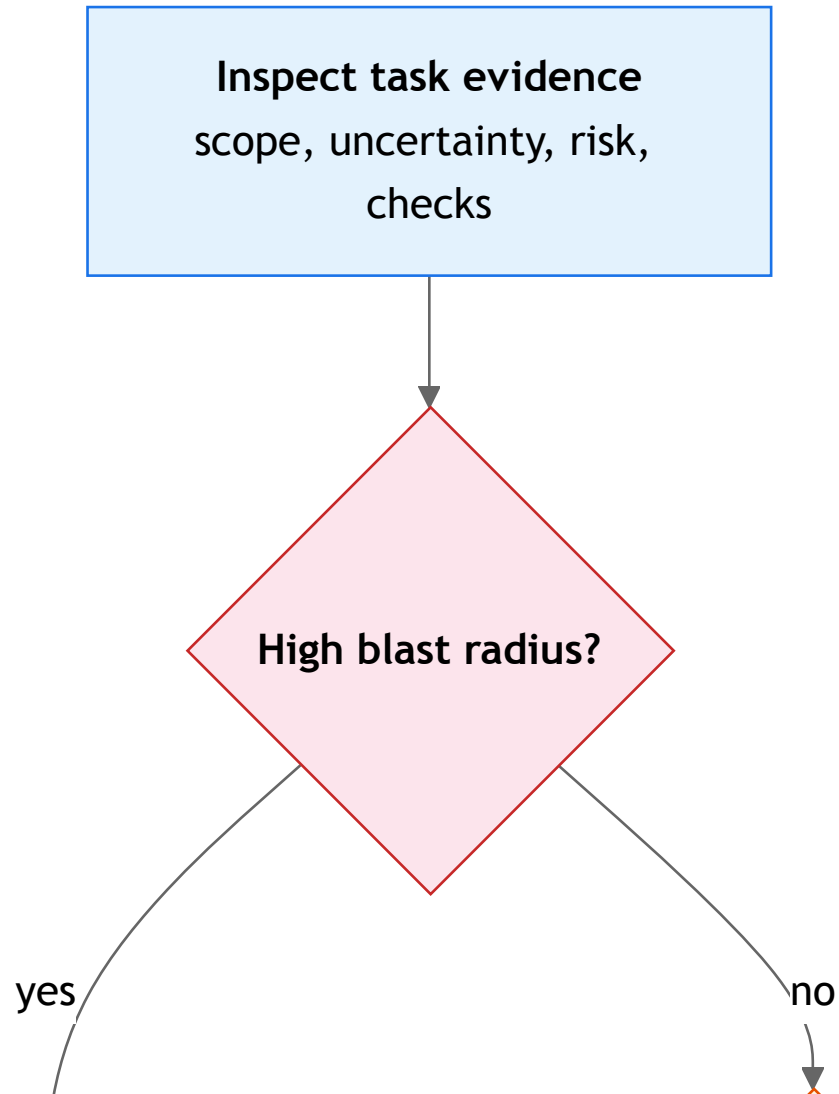
Effort controls response-token use across text, thinking, and tool calls.

Low or medium: Bounded work after evaluations show quality holds.

High: The documented default for difficult coding and agentic tasks.

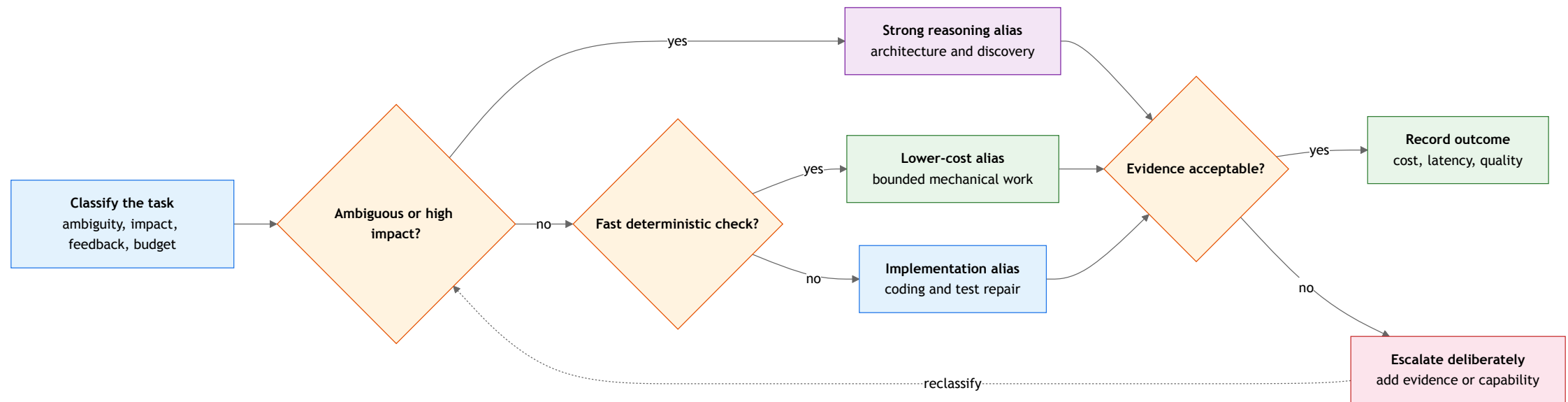
Higher levels: Long-running work when supported and justified by evidence.

Model Selection Decision Tree



Generic Model Routing Decision

REFERENCE ONLY



Four Task Shapes

Task	Dominant signal	Initial strategy
Update delivery copy	Local and reversible	Efficiency first
Add a carrier mapping	Specified and easy to check	Balanced or efficient
Diagnose duplicates	Uncertain and cross-service	Capability first
Change refund authorization	High blast radius	Strong model plus approval

Demo: Model Selection Matrix

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-model-selection-matrix.py`.

- Evidence score: four task signals drive the initial route.
- Model and effort: both choices appear in the record.
- Approval: irreversible work remains human-controlled.

Dated recommendations run offline; they do not call a model.

Demo: Routing Signals Measured From the Repo

From the companion repo, run `python3 day1/demos/demo-routing-signals-measured.py`.

- Blast radius: counted from real imports, not assigned by hand.
- Verification cover: real test methods in modules that import the target.
- Reversibility: read from the locked schema contract on disk.
- Three tiers: one rule, three measured inputs, three different answers.

Runs offline against the course codebase; no model is called.

Notebook: `day1/demos-notebook/demo-routing-signals-measured.ipynb`.

Inspect the Recommendation

The matrix is a governance aid, not a capability guarantee.

Ask whether each input came from repository evidence or assumption.

Check whether the recommended verification can detect a bad outcome.

Override the route when risk policy or organizational access requires it.

Optimize for Accepted Outcomes

Price the retries, review, and rework

Per-Token Price Is Incomplete

Track the cost of accepted work:

- Attempts and pass rate
- End-to-end latency
- Input and output tokens
- Human review time
- Rework and regression repair

A capable model can cost less per completion if it avoids repeated failure.

Compare under Controlled Conditions

Keep the task, prompt, tools, sandbox, time, and resource limits fixed.

Run multiple trials when behavior varies.

Grade repository outcomes rather than the agent's completion message.

Anthropic measured a six-point benchmark swing from infrastructure alone.

Use the Right Grader

Evidence	Suitable grader
Tests, compilation, schema, or static checks	Code-based
Instruction following or nuanced quality	Calibrated rubric or model-based
Security, architecture, or business risk	Human review

Combine graders when no single check covers the acceptance contract.

Escalation Needs a Signal

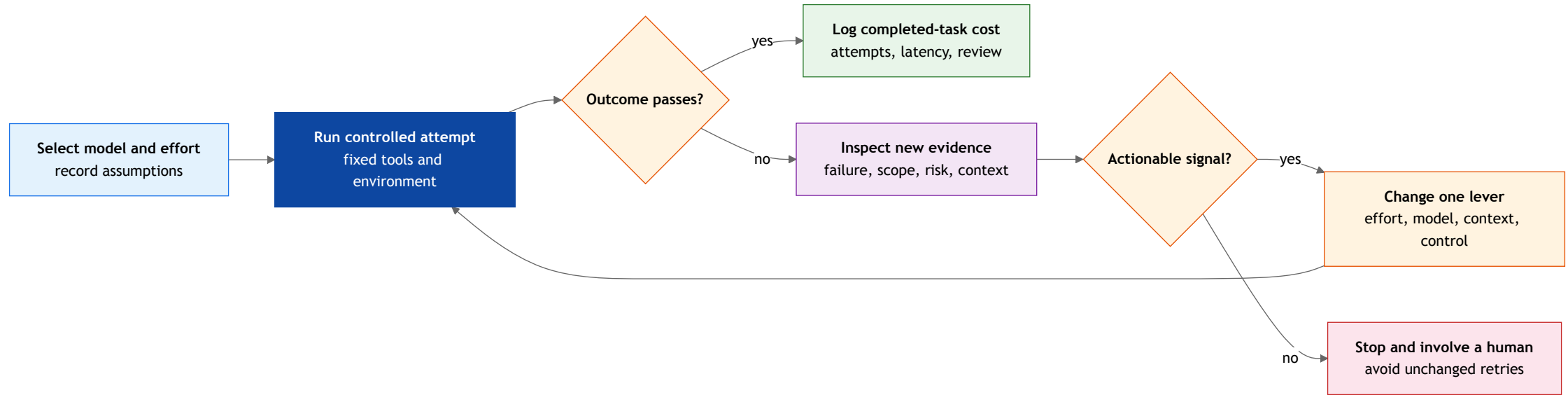
Escalate after a reproducible failure, unexpected scope, missing context, or new risk.

Change one relevant lever: effort, model, context, task boundary, or human control.

Do not repeat unchanged conditions and call the retry a strategy.

Stop when the trajectory provides no actionable evidence.

Evidence-Based Escalation Loop



Predict the Routing Decision

The focused test passes, but a full scan shows that input sanitization was removed. What should happen next?

- A) Stop for human security review
- B) Upgrade the model automatically
- C) Lower effort and retry
- D) Repeat the unchanged prompt

Decide privately first. Then compare reasons and produce one evidence-based answer.

Demo: Escalation From a Real Test Failure

From the companion repo, run `python3 day1/demos/demo-escalation-from-failure.py`.

- Injected regression: `escape()` is removed from the untrusted field.
- Real failure text: the `<script>` payload appears in the assertion output.
- Decision: the security failure stops execution for a human review.
- Same rule, repaired file: the decision changes because the evidence changed.

Edits a temporary copy only; no API key and no network needed.

Notebook: `day1/demos-notebook/demo-escalation-from-failure.ipynb`.

Explain the Security Stop

A - Stop for human security review

The repository-wide evidence shows a crossed security boundary. A stronger model does not grant authority to continue automatically.

After the file is repaired and the same security check passes, the evidence supports **KEEP**.

Debrief: Why would option B improve capability without resolving governance?

Demo: Evidence-Based Escalation

REFERENCE ONLY

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-evidence-based-escalation.py`.

- Focused failure: keeps the initial route while narrowing inspection.
- Cross-service scope: escalates capability deliberately.
- Security boundary: stops execution for human review.

Reference alternative: use this deterministic trajectory instead of the practical prediction sequence, not before it.

Selection Record

Capture the model, effort, date, entitlement, and fallback.

Record the task signals and controlled evaluation conditions.

Attach acceptance results, attempts, latency, review, and total cost.

Name the escalation trigger before execution begins.

Key Takeaways

1. Select models from task evidence rather than task wording alone.
2. Tune effort separately and validate every reduction against evaluations.
3. Compare cost per accepted task, including retries and review.
4. Hold the harness constant when comparing configurations.
5. Escalate on new evidence and preserve human control for high risk.

Ready for Kahoot Quiz 1?

Choose the route, then defend it with evidence.